

Project Notes: Elixir of Things DSL and Dashboard Extension

Part 0 - The Problem

Title: Elixir of Things - an IoT control and monitoring DSL

Problem description:

The problem is to describe and implement a distributed IoT system with a domain-specific language and generate the executable system from that description. The system includes nodes, sensors, actuators, a MQTT broker, and rule-based coordinators. The generated solution must run on embedded Linux devices and should also expose model data that can be visualized in a React dashboard.

This is a model-driven software development problem: one textual DSL model is transformed into runtime code for the Raspberry Pis and into JSON model files for the dashboard.

Part 4 - The Topic

The Topic: Data visualization

In this project, data visualization is the interactive React dashboard that displays the generated topology, topics, live MQTT messages, and active devices. The dashboard turns the DSL model into an inspectable visual representation of the IoT system.

Project Overview

This project is a small model-driven IoT platform. The DSL is used to describe a distributed embedded system with nodes, sensors, actuators, coordinators, topics, and rules. From that single source model, the generator produces the Elixir/Nerves code that runs on Raspberry Pis, and it also produces JSON files that the React dashboard can read.

The important idea is that the DSL is the source of truth. The dashboard is not manually configured. Instead, it reflects whatever the DSL generator emits. That gives a clean separation between the system model, the generated runtime code, and the visual monitoring layer.

Internal DSL vs External DSL

In exercise 3, the internal DSL was built in Elixir using macros, nested `do...end` blocks, and context stored in the process dictionary. That approach is useful when you want to extend an existing host language and reuse its syntax, runtime, and metamodel structures.

This project deliberately uses an external DSL instead. The reason is that the language needed its own dedicated syntax for IoT concepts such as `system`, `node`, `sensor`, `trigger`, `coordinator`, and `rule`. An external DSL is a better fit here because it:

- makes the domain vocabulary explicit and easier to read
- allows stronger syntax and semantic validation at the language level

- keeps the IoT model independent from Elixir syntax
- supports separate code generation targets, such as Elixir/Nerves and dashboard JSON

The internal DSL from exercise 3 is therefore the implementation comparison point, but the external DSL is the better design for this project because the language itself is the product.

1. What the DSL models

The language describes an IoT system in a declarative way. A system contains:

- one MQTT broker
- multiple nodes, each with an IP address
- topics used for communication
- sensors that publish data
- actuators that react to messages
- coordinators that evaluate rules and forward control messages

This matches the typical architecture of an event-driven IoT system: sensors observe the environment, topics carry events, coordinators interpret them, and actuators change the physical world.

Why a DSL is useful here

A domain-specific language is a better fit than general-purpose code when the problem has a stable structure and repeated concepts. In this project, the repeated concepts are not arbitrary algorithms; they are domain entities such as temperature sensors, MQTT topics, GPIO pins, and rule conditions. The DSL makes those concepts explicit, readable, and easier to validate.

The course material on MDSD, meta-modeling, parsing, and code generation fits this project well because the language is not just a notation. It is the starting point of a full transformation pipeline.

Declarative or Imperative

This external DSL is declarative.

It describes what the IoT system should contain and how devices should react, but it does not describe the step-by-step control flow of the implementation. The user states facts such as which node hosts which sensor, which topics exist, and which rule should fire when a condition holds. The generator then turns those declarations into executable Elixir/Nerves code.

That is different from an imperative language, where the programmer would specify the exact sequence of operations. Here, the focus is on the desired system structure and behavior, so declarative is the correct classification.

Short summary - declarative DSL

The DSL is declarative because it describes the IoT system as facts and rules, and the generator turns that model into executable behavior.

2. Grammar notes

The grammar is defined in Xtext and describes both the syntax and the structure of the model. The file you pointed at, `ElixirOfThings.xtext`, is this grammar definition. The actual generator is separate and lives in the Xtend file `ElixirOfThingsGenerator.xtend`. The main rules are:

- `System` as the root element
- `Broker`, `Node`, and `Topic` as the infrastructure layer
- `Sensor`, `Trigger`, and `TriggerAction` for event production
- `Actuator` and `OnMessage` for event consumption and physical output
- `Coordinator`, `Rule`, and `RuleAction` for rule-based control

The grammar also defines enumerations for sensor types, actuator types, QoS, operators, states, and time units. That is important because it constrains the language to valid IoT concepts instead of free-form text.

Concrete syntax overview

The grammar follows a readable block structure with keywords and braces. A program starts with `system`, then declares the broker, the nodes, the topic catalog, and finally the device definitions. A typical system therefore reads like a specification rather than an algorithm.

Some important syntax choices are:

- `nodes+=Node+` means the system must contain at least one node.
- `topics+=Topic*` means topics are optional, but usually present.
- `triggers+=Trigger*`, `messages+=OnMessage*`, and `rules+=Rule*` allow repeated nested elements.
- `deployedOn=[Node]` is a cross-reference, so sensors, actuators, and coordinators point to an existing node instead of duplicating node data.
- `subscribeTo+=[Topic] (',' subscribeTo+=[Topic])*` uses resolved topic references rather than raw strings.
- `TriggerAction` and `RuleAction` also point to `[Topic]`, so publishing targets are model elements.

The current grammar also differs from the earlier course example in one important way: topics, subscriptions, and publishing targets are model elements rather than free-form topic strings. That makes the model more structured and gives the generator stronger references to work with.

Domain modeling choices

The grammar makes the IoT domain explicit by separating concerns into different rules:

- `Broker` stores the MQTT endpoint.
- `Node` stores the system node name.
- `Topic` stores the topic name, payload string, and QoS.
- `Sensor` stores type, GPIO pin, sample rate, deployment node, and triggers.
- `Actuator` stores type, GPIO pin, deployment node, subscriptions, and message reactions.
- `Coordinator` stores subscriptions and rule logic.

That separation is useful because the generator can treat each concept differently when producing code and dashboard metadata.

EBNF cheat sheet

Yes — the Xtext grammar for this project is the EBNF-style syntax definition for the DSL, but Xtext extends plain EBNF with model features such as assignments, datatype rules, and cross-references.

Quick notation guide:

- `A B` means sequence: `A` followed by `B`
- `A | B` means choice: either `A` or `B`
- `A*` means zero or more repetitions
- `A+` means one or more repetitions
- `A?` means optional
- `( ... )` groups parts together
- `'text'` or `"text"` matches a literal keyword or symbol
- `[Type]` is a cross-reference to another model element
- `name=Rule` assigns the parsed value to a feature
- `nodes+=Node+` means append one or more `Node` objects to a list feature
- `hidden(WS, SL_COMMENT, ML_COMMENT)` controls whitespace and comments

How to read the project grammar with that cheat sheet:

- `system ...` starts the root rule
- `broker=Broker` stores one broker object in the system model
- `nodes+=Node+` requires at least one node
- `topics+=Topic*` allows an optional topic list
- `deployedOn=[Node]` links to an existing node
- `subscribeTo+=[Topic]` links to one or more existing topics
- `NumExpr`, `NumMul`, and `NumAtom` encode precedence for arithmetic expressions
- `BoolOr`, `BoolAnd`, and `BoolNot` encode precedence for boolean expressions

So for exam notes, it is fine to say that the DSL is specified in Xtext using an EBNF-like grammar, with Xtext-specific extensions for AST creation and name linking.

Short summary - Xtext grammar

Xtext uses an EBNF-like grammar, but it also adds assignments, cross-references, and model building, so it is more than plain EBNF.

Structural observations

The grammar mixes two useful ideas:

- concrete syntax, which is the exact text the user writes
- abstract syntax, which is the structured model the generator consumes

That is the standard compiler/DSL pipeline: parse text into a model, then transform the model into another representation. In MSDS terms, the grammar defines the concrete syntax and the generator consumes the resulting abstract syntax.

Expression handling

The DSL supports arithmetic and boolean expressions.

- Arithmetic expressions are used for sample rates and durations.
- Boolean expressions are used in coordinator rules.

This is an important design choice because it lets the user write values such as `2 * 5 sec` or conditions such as `temperature/high and not motion/detected` instead of hard-coding those details in the generated Elixir.

The grammar also keeps rule conditions close to the domain language used by the IoT system. Instead of exposing low-level boolean flags, it lets the author write topic paths and logical operators directly in the DSL.

From a language-theory point of view, expressions are where the DSL becomes more than a list of objects. They define an evaluation model:

- the parser builds a tree-shaped abstract syntax model
- operator precedence is encoded by the grammar structure
- associativity is handled by the repeated subrules
- parentheses let the user override the default grouping

That means the DSL does not just store values; it stores computation. For arithmetic, the tree can represent nested operations such as `2 * (5 + 1)`. For boolean rules, the tree can represent layered logic such as `a and b or not c` without ambiguity.

The generator then interprets these trees. In practice, that gives three benefits:

- static expression parts can be reduced early
- the generated Elixir code becomes simpler
- the runtime only has to evaluate what cannot be known at generation time

This is why the expression rules are interesting from an MDSD perspective. They are a small example of partial evaluation: the generator can fold constants like numeric rates and durations before producing code. That is especially useful for sample rates, because the final runtime usually wants a concrete millisecond value rather than an arithmetic formula.

The same idea applies to boolean rules. The generator can preserve the logical structure of the condition while still translating operators into the target language. In other words, the DSL expression is a source-level representation of logic, and the generator performs a controlled translation instead of a textual copy.

There is also a type aspect here. Numeric expressions and boolean expressions are different categories, so the grammar keeps them separate. That prevents invalid mixes such as putting a boolean condition where a duration is expected. In a fuller language implementation, validation could also check domain constraints like non-negative rates, non-zero divisors, or references to only existing topics.

So the theory behind expression handling is:

- syntax defines how expressions are written
- the metamodel defines how they are structured
- validation checks whether they are meaningful
- generation turns them into executable target code

Left recursion

Left recursion is a grammar pattern where a rule refers to itself as its first symbol, for example `Expr -> Expr + Term`. That is a problem for many top-down parsers, including Xtext-based grammars, because the parser can recurse forever before consuming any input.

In this project, the grammar already avoids left recursion in the important expression rules. The numeric rules use the standard Xtext expression pattern:

- `NumExpr -> NumMul ((+ | -) NumMul)*`
- `NumMul -> NumAtom ((* | /) NumAtom)*`

and the boolean rules use the same layered pattern:

- `BoolOr -> BoolAnd (or BoolAnd)*`
- `BoolAnd -> BoolNot (and BoolNot)*`
- `BoolNot -> not BoolAtom | BoolAtom`

These are not left-recursive productions because the recursive part is not the first symbol in the rule. That makes the grammar parser-friendly while still allowing the generator to evaluate the resulting AST recursively.

So the fix is not in the generator; the fix is in the grammar design. The generator can traverse recursive trees safely, but the grammar itself must avoid left-recursive productions.

Short summary - recursion

Left recursion must be avoided in the grammar, and Xtext solves it by using layered precedence rules instead of rules that call themselves first.

Metamodel programming

The course slides on metamodel programming fit this project directly. Xtext infers a metamodel from the grammar, so each rule becomes a model class and each alternative becomes a subtype where needed. Cross-references such as `[Node]` and `[Topic]` link uses of names to their declarations, which is why scoping matters so much.

3. Generator notes

The generator is implemented in Xtend and walks the DSL model node by node. For every node in the system, it creates a corresponding generated folder under `src-gen/lib/<node_name>/`.

The generator emits these main artifacts:

- `application.ex` for the OTP supervision tree and MQTT connection setup
- one sensor module per sensor
- one actuator module per actuator
- `mqtt_handler.ex` for nodes that host a coordinator
- `coordinator.ex` for the rule engine

Why this is a good generation strategy

The generator is not producing one giant program. Instead, it maps the model to small OTP components. That reflects the Elixir/Nerves style well, because the runtime is built from supervised, isolated processes. This is a

good example of model-to-text transformation and component-based code generation.

Key generator behavior

- It converts node names into Elixir module prefixes.
- It evaluates numeric expressions at generation time, so durations and sample rates become concrete millisecond values in the generated code.
- It uses different runtime strategies for different sensor types, such as polling for temperature and interrupt-driven handling for motion.
- It builds a coordinator state map from subscribed topics and evaluates rules against that state.
- It also generates dashboard metadata files, `dashboard/topology.json` and `dashboard/topics.json`, from the same model.
- It derives topic producers and consumers by inspecting actual model references, not by reading raw strings from the DSL text.

Helper functions in the generator

The Xtend generator also contains several helper functions that keep the main generation code small and readable:

- `toCamelCase` converts DSL names like `temp_sensor` into Elixir module names such as `TempSensor`.
- `toElixirOp` maps DSL comparison operators to Elixir operators like `>`, `<`, `>=`, `<=`, and `==`.
- `topicStr` extracts the topic string from a `Topic` model element.
- `evalNumExpr` evaluates arithmetic expressions in the DSL model before code generation.
- `sampleRateToMillis` converts a `SampleRate` value into milliseconds.
- `durationToMillis` reuses the same conversion for timed actuator actions.
- `generateBoolExpr` recursively turns the boolean expression tree into an Elixir condition.
- `uniqueStateKeys` collects the topic prefixes that become coordinator state keys.
- `jsonString` escapes strings for JSON output.
- `findProducers` and `findConsumers` analyze the model to populate topic relationships in the generated dashboard JSON.

These are not separate external services. They are internal generator helpers that transform the resolved DSL model into executable code and dashboard metadata.

The generator does not push this work outside the Xtend codebase. The transformation logic is implemented where it belongs: in the generator, backed by small helper methods that keep the code readable. In this project, that logic is complex enough to be interesting, but not so complex that it needs a separate external function layer.

Theoretical point

This is a common compiler technique: partial evaluation at generation time. Instead of leaving every expression to runtime, the generator resolves static arithmetic where possible. That reduces runtime complexity and makes the generated code simpler.

The same idea appears in MDSD lectures as moving work from execution time to generation time when the model already contains enough information to compute a result safely.

The generator also performs a form of program analysis when it evaluates arithmetic expressions early. `evalNumExpr` is effectively constant folding for sample rates and durations: if the DSL says `2 * 5 sec`, the generator writes the resulting millisecond value into the output code instead of leaving the expression for runtime.

External functions

The lecture slides mention external functions as a way to handle logic that is too complex or too language-specific to express directly in the grammar. This project does not need that mechanism. The generator already contains the required logic for operator mapping, numeric evaluation, topic analysis, and JSON creation, so no separate external function layer is necessary.

Short summary - generator and dashboard

The generator is where the model gets translated into code, and it already performs useful compile-time work like constant folding and topic analysis.

4. Why the JSON files were needed for the dashboard

The React dashboard in `dashboard/` is an individual extension on top of the DSL project. It needs a data model it can read quickly in the browser, so the DSL project was extended to generate JSON files.

The dashboard loads:

- `/dashboard/topology.json`
- `/dashboard/topics.json`

These files provide the information the UI needs to show the system state even before MQTT traffic starts flowing.

What the dashboard uses them for

- to show the system name and broker address
- to render nodes, sensors, actuators, and coordinators
- to display topic ownership and topic relationships
- to highlight active actuators and live message activity

This is a good example of decoupling: the dashboard depends on generated model data, not on the raw DSL parser or the generated Elixir source code.

5. Notes about the dashboard extension

The dashboard is a separate React app that acts as a live visualization layer for the DSL project. It reads the generated JSON model and combines it with live MQTT messages.

From the codebase structure, the dashboard is doing three things:

1. loading the generated topology and topic metadata
2. connecting to the MQTT broker over WebSocket
3. showing a topology view plus a message feed and node inspection panel

That means the React app is not just a static report. It is an operational view of the system.

Good software engineering idea behind the extension

The UI is built from generated data plus runtime events. That keeps the visualization layer independent from the execution layer. If the DSL model changes, the dashboard adapts by reading the new JSON, instead of requiring manual changes in the frontend.

The generator is also doing model analysis here: it computes which devices produce and consume each topic, so the dashboard can show relationships instead of only raw labels.

That is also why data visualization is a good topic for this project: the dashboard does not invent new domain data, it presents generated and live system data in a way that is easier to understand than the raw DSL or raw MQTT traffic.

6. Theory that applies to this project

Model-driven engineering

This project is a clear model-driven engineering example. The DSL is the model, the Xtend generator is the transformation step, and the Elixir/Nerves projects are the generated execution artifacts.

Separation of concerns

The project separates:

- domain modeling in the DSL
- execution in Elixir/Nerves
- visualization in React
- communication through MQTT

That separation makes the system easier to reason about and easier to extend.

Reactive and event-driven systems

IoT systems are naturally reactive. Sensors emit events, rules respond to events, and actuators perform actions. MQTT is a good match because it is lightweight and topic-based.

Actor-style concurrency and supervision

Elixir/Nerves runs on the OTP model, where processes are supervised and isolated. That matters in embedded systems because a sensor process, actuator process, or coordinator process can fail without taking the whole system down.

Syntax versus semantics

The grammar defines how programs look. The generator defines what they mean in practice by translating them into sensor polling, interrupt handling, MQTT routing, and rule evaluation.

Validation and scoping

Validation in this project happens at two levels. First, the grammar itself enforces the structure of valid programs: a system must have a broker and at least one node, sensors and actuators must be deployed on a node, and topics, subscriptions, and rule conditions must follow the DSL shape. Second, model-based validation can check semantic constraints such as whether a referenced node or topic exists, whether a sample rate is meaningful, or whether a rule contains a valid condition.

Scoping is the mechanism that makes cross-references work. In the grammar, constructs like `deployedOn=[Node]`, `topic=[Topic]`, and `subscribeTo+= [Topic]` are not just text matches. Xtext resolves them against the visible model elements in the current system. That means the DSL can refer to actual nodes and topics instead of duplicating names everywhere.

This is important for the generator because it can trust the model references it receives. The generator does not need to re-parse strings to find node or topic relationships; it can iterate over resolved model objects and use them directly when generating Elixir code and dashboard JSON.

In exam terms, validation ensures the model is correct, and scoping ensures names are resolved in the correct context.

Liveness of the dashboard

The dashboard reaches **Level 4: informative, significant, responsive, and live**.

- It is informative because it visualizes the generated topology, topics, nodes, sensors, actuators, and live messages.
- It is significant because the visualization is based on the actual generated model and MQTT topic relationships.
- It is responsive because the UI updates when the user selects nodes and when the active actuator state changes.
- It is live because it connects to the MQTT broker over WebSocket and updates the message feed in real time.

It is not Level 5 or Level 6. The dashboard does not predict user actions, and it does not create itself from developer intention. The executable part is the generated Elixir/Nerves system; the dashboard is the live observation layer on top of it.

Short summary

The dashboard is Level 4: it is live, responsive, and based on real model and MQTT data, but it does not predict or self-generate.

7. Short project summary

This project demonstrates a complete DSL pipeline for an IoT system:

- design a textual language for distributed devices
- parse it into an abstract model
- generate runnable Elixir/Nerves code
- generate JSON metadata for the dashboard
- visualize the system and monitor MQTT traffic in React

In short, the DSL is the specification layer, the generator is the transformation layer, and the dashboard is the observation layer.

Final exam line

This project is a declarative Xtext DSL that models an IoT system, generates Elixir/Nerves runtime code and dashboard JSON, and visualizes the live MQTT-based system in React.